Task-Aware Progressive Retrieval Attention: Virtualizing Interleaved Agent Context by Execution Scope

PRA Research Program

August 2026

Abstract

Progressive Retrieval Attention (PRA) has largely been studied under an implicit single-task assumption: one physical session is treated as one coherent retrieval space. Long-running agents instead multiplex interleaved tasks. We introduce task-aware PRA, in which harness-authoritative task identity and dependency relations restrict candidates before ordinary PRA discovery and Paper-7 progressive materialization. A model-free study over 80 workflows first isolates the scope mechanism. We then hold a production native-Q/K hybrid router fixed across 15 naturalistic typed-record cases and vary only task admission. Structural scope raises exact evidence availability from 33.3% to 93.3%, removes 75.0% session-wide cross-task contamination, and reduces requested native tokens from 234.3 to 93.5 per case. With selected exact records made visible, frozen Qwen3-0.6B answer accuracy rises from 13.3% to 66.7%; full task scope reaches 73.3% versus 20.0% for the full interleaved session. Direct frozen native-memory injection yields 0.0% accuracy despite successful evidence routing, separating a consumption failure from the scope result. Join sweeps show the remaining capacity boundary. The contribution is therefore a replayable task-scope layer and evidence that task structure improves both retrieval allocation and downstream behavior when the selected detail is consumable, not a claim that task planning or frozen native-memory integration is solved.

1 Dependency Gate and Scope

Implementation begins only after the stable typed-record, addressability, progressive-materialization, and cursor mechanisms from Paper 7 are integrated into the SDK path used by Paper 8. Paper 8 is restricted to one physical session containing multiple logical tasks. Separate-session subagents are Paper 9 and cross-session memory is Paper 10.

Paper 8 does not introduce a parallel memory representation. It inherits Paper 7 typed records and adds task provenance and workflow structure as an addressing layer.

2 Motivation

Let a physical session stream be

$$H = (r_1, \ldots, r_n).$$

In an agent session, records may belong to multiple tasks and be heavily interleaved. Physical recency is therefore not equivalent to execution relevance. The core change is from

$$P(r \mid q)$$

to

$$P(r \mid T, q, G_T),$$

where T is the active task and G_T is the harness-authoritative task/workflow structure.

The key architectural decomposition is

$$\boxed{\text{Acquire Task Structure} \rightarrow \text{Select Task Scope} \rightarrow \text{PRA Discover} \rightarrow \text{Materialize}}$$

rather than treating all four as one learned retrieval problem.

3 Responsibility Boundary

Harness/session manager. The harness is the source of truth for the full session event stream, canonical task descriptors, task status, dependency graph, tool execution, authoritative outcomes, workspace state, result/evidence mappings, authorization, and validation/commit of task transitions. Model-generated task operations are proposals until validated.

PRA/model runtime. The PRA runtime owns a replayable projection: task-aware candidate partitions, indexes, task facets, routing, record/chunk selection, bounded native K/V, and active view/materialization state. PRA is not a workflow engine.

Harness and model runtime may execute on different machines. Require stable session/task/record IDs, monotonic event sequence, versions, idempotent replay, snapshots, invalidation, and deterministic reconstruction.

4 Paper-7 Record Inheritance

Let a Paper 7 typed record be

$$R_7 = (id, \tau, P, A, V, B),$$

with stable identity, type, provenance/policy, address views A, current visible view V, and lossless backing state B.

Paper 8 extends this with task provenance:

$$R_8 = (R_7, \Pi_T),$$

where Π_T may include task ID, producing task, consuming task(s), result/evidence role, and event sequence.

Paper 8 therefore studies *which task-scoped records should be candidates*. It reuses ordinary PRA to discover records/chunks and Paper 7 to control detail/materialization depth.

5 Task and Workflow Records

The canonical harness state should expose a compact model/runtime projection such as

$$\begin{aligned} TaskState = \{&id, version, status, description, constraints, parent,\\ &dependencies, blockers, evidence, outputs, result\_ref,\\ &created\_seq, updated\_seq\}. \end{aligned}$$

Status v1 is

$$\{\texttt{pending}, \texttt{active}, \texttt{blocked}, \texttt{completed}, \texttt{cancelled}\}.$$

Prefer `result_ref` over embedding large result summaries directly in authoritative task state. The referenced result is a Paper 7 typed record with compact view, address views, and full backing state.

6 Scope, Discovery, and Materialization

For active task T and query q:

$$C(T, q) = M(T) \cup S(T) \cup R(T, q),$$

where $M(T)$ is mandatory control state, $S(T)$ is explicit structural closure, and $R(T, q)$ is ordinary PRA retrieval over records admitted by scope policy.

Define a bounded structural closure

$$Closure(T) = \{T,\ Parent(T),\ UnresolvedDeps(T),\ Blockers(T),\ RequiredEvidence(T)\}.$$

Siblings and unrelated tasks are excluded unless explicitly related or retrieved through fallback.

The full architecture becomes

$$\boxed{ScopePolicy(T, G_T) \to PRASelection(q) \to MaterializationPolicy(R)}.$$

Paper 8 primarily studies the first term. Existing PRA studies the second. Paper 7 supplies the third.

7 Scope Breadth × Materialization Depth

Task-aware context management is naturally two-dimensional.

Scope breadth.

$$TaskLocal \to StructuralClosure \to RelatedTasks \to SessionGlobal.$$

Materialization depth.

$$Index/Gist \to Compact \to SelectedDetail \to Full/NativeKV.$$

Thus a task policy is not one scalar "context size" but a pair

$$\pi = (B_{\text{scope}}, B_{\text{detail}}).$$

Paper 8 should test whether a bounded budget is better spent on fewer task-relevant records at greater detail or on broader task scope at shallower detail.

8 Task Completion as Context Demotion

Task completion is not destructive summarization. It is a demotion boundary:

$$ExecutionHistory(T) \to V_{\text{completed}}(T) + A(B_T) + B_T.$$

The compact task result remains visible/indexable, full task evidence remains addressable, and exact backing state remains recoverable through Paper 7 mechanisms.

A completed task can therefore move from hot/full active state to compact/index state while retaining exact provenance and selective expandability.

9 Task Transitions

Harness-authoritative events include

`TASK_ACTIVATE`, `TASK_BLOCK`, `TASK_RESUME`, `TASK_COMPLETE`, `TASK_CANCEL`.

In oracle phases these events deterministically recompute scope and update the mandatory set. They also trigger promotion, demotion, cache changes, and materialization changes. Model callbacks are not required to establish the mechanism.

10 How Task Structure Is Acquired

Task-aware PRA must not assume every model can reliably create tasks online. We therefore separate task-structure acquisition from task-aware retrieval.

10.1 Oracle Structure

The harness receives the correct task graph. This is the mechanism ceiling and the first required experiment.

10.2 Single-Task Baseline

The whole request remains one task. No decomposition overhead and no task-specific gain.

10.3 Preflight Two-Pass Decomposition

Before long execution/tool use begins, the harness predicts whether task splitting is warranted. If so, it performs a dedicated decomposition model call:

$$Q \to \text{complexity gate} \to \text{decomposition pass} \to \hat{G}_T \to \text{execution pass}.$$

Two output protocols are required:

(a) JSON/schema-constrained output for models with reliable structured-output support;

(b) simple Markdown task grammar with deterministic parser for models that are unreliable with JSON.

The resulting task records exist before the long execution context begins.

10.4 Online Tool-Managed Tasks

The execution model receives always-available task-management primitives such as

`task_create`, `task_update`, `task_link`, `task_complete`, `task_get_state`.

This is elegant but model-capability dependent and may create structure late.

10.5 Hybrid

Preflight decomposition initializes the task graph; online tools may mutate it when execution discovers unforeseen work.

11 Task-Structure Acquisition Baselines

Use:

$$\begin{aligned} T0 &= \text{ORACLE}, & T1 &= \text{SINGLE}, \\ T2 &= \text{PREFLIGHT-JSON}, & T3 &= \text{PREFLIGHT-MD}, \\ T4 &= \text{ONLINE-TOOLS}, & T5 &= \text{HYBRID}. \end{aligned}$$

Task decomposition should be evaluated both structurally and by downstream utility. A decomposition may differ from a reference graph yet still support correct task-aware PRA.

12 Workflow Complexity as a Primary Variable

Task complexity must be an explicit experimental axis rather than an incidental benchmark property.

Define workflow families:

C0: Atomic. One task, one coherent execution scope.

C1: Linear sequential chain.

$$T_1 \to T_2 \to \cdots \to T_n.$$

Measure number of steps/tasks and dependency depth.

C2: Independent parallel set.

$$\{T_1, \ldots, T_n\}, \qquad E = \varnothing.$$

No dependencies; interleaving creates contamination without structural evidence flow.

C3: Fork. One task produces multiple independent children.

C4: Join. Multiple predecessor results are required by one downstream task:

$$T_1, T_2, \ldots, T_k \to T_j.$$

This tests whether explicit structural closure recovers all required upstream outputs without reopening unrelated history.

C5: General DAG. Arbitrary acyclic dependencies with multiple forks/joins.

Track:

$$N_T = \text{task count}, \quad N_E = \text{dependency-edge count}, \quad D = \text{critical-path depth},$$

$$W = \text{maximum parallel width}, \quad J = \text{join count}, \quad F = \text{fork count}.$$

Also track the number of execution steps/records per task, because a five-task workflow with one event/task is not equivalent to five long-running tasks.

13 Interleaving Complexity

Independently vary:

- interleaving frequency / task switches;
- resumption distance in tokens and records;
- semantic similarity among tasks;
- shared entities;
- records per task;
- completed-task history;
- dependency-result delay;
- active/hot distractor state.

A particularly important condition is *hot wrong-task contamination*: switch from Task A to a semantically similar Task B while A's detailed/native K/V is still hot. Measure whether task-aware scope correctly demotes A.

14 Cold, Warm, and Hot Task Resumption

Distinguish:

Cold resume: task backing state exists but native encoding/cache is absent;

Warm resume: task encoding is cached but inactive;

Hot resume: relevant task K/V remains accelerator-resident.

Report switch/resume cost as

$$Cost_{\text{switch}} = Cost_{\text{demote}} + Cost_{\text{scope}} + Cost_{\text{promote/materialize}}.$$

Measure K/V reused, demoted, newly materialized, TTFT, and task resumption accuracy.

15 Task-Resumption Curve

A headline experiment should vary intervening task distance:

$$x = \text{intervening records/tokens/task switches},$$

$$y = \text{resumed-task evidence recall/task success}.$$

Compare ordinary session-wide PRA against task-local, structural, and adaptive task-aware PRA at matched active-K/V budgets.

16 Primary PRA Baselines

Use a simplified main ladder:

$$P0 = \text{FULL/TRUNCATION},$$

$$P1 = \text{PRA\_SESSION},$$

ordinary PRA over the interleaved session;

$$P2 = \text{PRA\_TASK\_LOCAL},$$

hard active-task scope;

$$P3 = \text{PRA\_TASK\_STRUCT},$$

active task plus explicit structural closure;

$$P4 = \text{PRA\_TASK\_ADAPTIVE},$$

structural closure plus controlled widening to related/session-global scope.

Task tags ignored and alternative routing/materialization policies remain ablations rather than primary reader-facing systems.

17 Task-Structure Acquisition Evaluation

For PREFLIGHT/ONLINE/HYBRID measure:

- decomposition-needed gate precision/recall;
- task count error;
- missing/duplicate task rate;
- dependency-edge precision/recall/F1;
- join/fork recovery;
- constraint/result-flow preservation;
- JSON/Markdown parse and validation failure;
- decomposition tokens/latency/model calls;
- downstream task-aware PRA success using generated structure.

Always separate graph-quality metrics from downstream usefulness.

18 Task-Aware PRA Evaluation

Hold required evidence constant while varying:

$$N_T,\ N_E,\ D,\ W,\ J,\ F,$$

interleaving, resumption distance, semantic similarity, records/task, and completed history.

Critical controls:

- unrelated distractors;
- semantically similar tasks;
- shared entities;
- delayed dependency outputs;
- stale/wrong task tags;
- missing task metadata;
- old-task resumption;
- hot previous-task K/V;
- join tasks requiring several predecessor outputs;
- parallel independent tasks where structural closure should remain narrow.

19 Metrics

Quality:

- relevant-record recall/precision;
- cross-task contamination;
- dependency/evidence recall;
- join completeness;
- required-action/task completion;
- constraint/dependency/state errors;
- stale-task errors.

Systems:

- active native tokens/KV;
- CPU/backing bytes;
- K/V promoted/demoted/reused;
- peak GPU memory;
- task-switch/resume TTFT;
- routing/materialization latency;
- tokens/FLOPs/cost per successful task and session.

Structure acquisition:

- extra round trips;
- planning tokens;
- parse failures;
- task graph fidelity;
- downstream benefit.

20 Implementation

The reference implementation extends the inherited typed-record substrate rather than introducing a task-specific memory store. `TaskState` records the status, description, constraints, parent, dependencies, blockers, evidence, outputs, and a compact `result_ref`. `TaskEvent` supplies a stable event ID, monotonic sequence, event type, expected task version, and mutation payload. `TaskGraph.apply` rejects stale versions, missing dependencies, self-links, and cycles; duplicate event IDs are idempotent. A serialized `TaskDescriptor` is therefore a cacheable snapshot, while the event stream remains replayable.

Typed context records retain their Paper-7 identity and views. Paper 8 adds `TaskProvenance` in selection provenance: owning task, optional producer and consumers, relation type, and physical event sequence. `TaskScopeSelector.partition` emits task and record IDs independently of any in-process model. The historical mechanism study applies a frozen lexical/recency ranker after this partition; the production study instead constructs the inherited Paper-7 native-Q/K index only over admitted exact backing records. `TaskWorkingSet` distinguishes `COLD`, host-encoded `ENCODED_HOST`, and accelerator-active `DEVICE_HOT`. Backing availability no longer implies warm encoding, and device demotion does not discard reusable host encoding.

The SDK substrate includes an abstract `SessionService`. `InMemorySessionService` provides thread-safe ephemeral state. `LocalSessionService` stores atomic, user-hashed JSON manifests on local disk and validates optimistic session versions. Both resolve exact sessions by `user_id` and `session_id`, or the most recently updated session for a user. Each session contains an ordered typed-record stream, the task descriptor, and replay events. Model-cache state is deliberately absent from durable session state because it is reconstructible. Persistent replay preserves admitted tasks, admitted records, and typed task provenance for all 15 production cases after the model cache is discarded.

The preflight control surface implements a cheap complexity gate and deterministic JSON and Markdown parsers. Parsed plans are validated through the same DAG implementation. No model-produced plan becomes authoritative merely because it parses.

21 Oracle Mechanism Experiment

We evaluate scope before task acquisition, following the causal order in Section 26. The generator creates semantically confusable records for atomic, linear, parallel, fork, join, and general-DAG workflows. The reported primary sweep uses parallel, linear, join, and DAG families; task counts $N_T \in \{2, 4, 8, 16\}$; six records per task; and seeds $\{11, 23, 37, 53, 71\}$. Records are physically interleaved. One unrelated evidence record is made maximally recent in contamination conditions. The active query uses the same shared vocabulary across tasks, so scope rather than a task-name lexical shortcut must separate evidence.

Within-scope ranking is fixed for every policy. For each record it counts query-term overlap and breaks ties by physical event sequence. The record budget is identical across policies. Relevant evidence is the active task for independent workflows and the structural closure for dependent workflows. We report record recall, useful precision, cross-task contamination, join completeness, candidate and active-token counts, scope latency, and working-set promotion/demotion. The implementation is ordinary Python and latency is diagnostic, not a serving claim.

The scope–detail experiment uses a fixed 192-token materialization allowance. Index, compact, selected, full, and native-K/V depths receive explicit per-record costs of 4, 12, 24, 48, and 64 tokens. We report both charged materialization tokens and logical backing text, avoiding the false equivalence between a compact visible view and its exact stored payload.

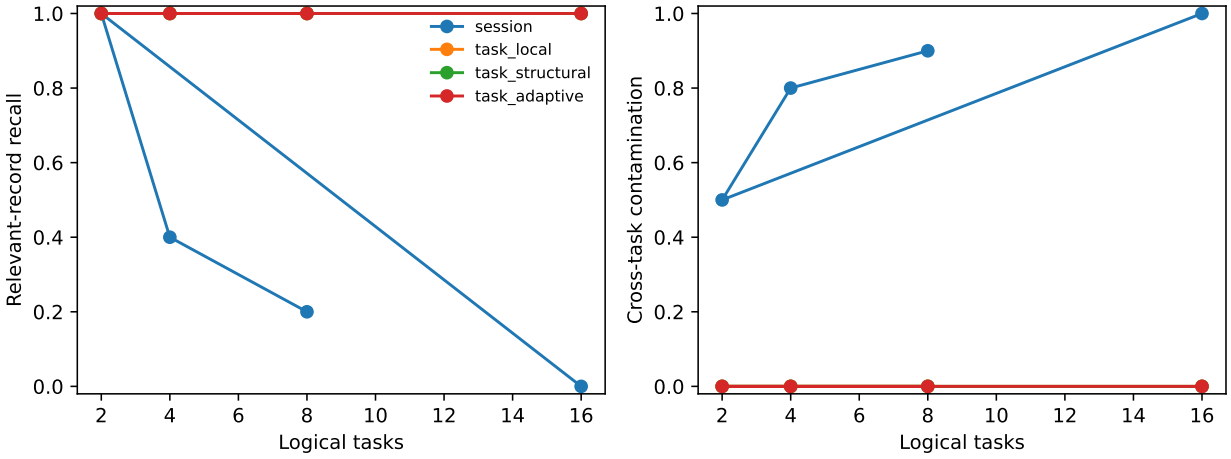

Figure 1: Oracle parallel-workflow scaling under a matched record budget. Session-wide ranking increasingly selects recent, semantically similar records from the wrong task. Task-aware admission preserves active-task recall and removes cross-task contamination in this controlled construction.

Table 1: Sixteen-task parallel condition, mean over five seeds. R is relevant-record recall, P useful precision, and C cross-task contamination.

Policy	R	P	C
PRA_SESSION	0.00	0.00	1.00
PRA_TASK_LOCAL	1.00	0.50	0.00
PRA_TASK_STRUCT	1.00	0.50	0.00
PRA_TASK_ADAPTIVE	1.00	0.50	0.00

22 Results

Figure 1 shows the central mechanism result. Averaged across the four parallel task counts, session-wide recall is 0.40 and contamination is 0.80. All three task-aware policies reach 1.00 recall and zero contamination. At 16 tasks the contrast is complete: session-wide recall is zero and contamination is one; every task-aware scope has recall one and contamination zero. Each selected set contains two records and 42 serialized visible tokens, so the difference is admission rather than a larger active context.

Hard isolation is insufficient when work depends on earlier tasks. Table 2 reports the eight-input join. Local scope sees only the active join task and recovers one of eight required records. Structural closure admits all explicit predecessors and recovers every input. Session scope also succeeds in this construction because the budget equals the relevant set, but it considers the complete session rather than using known graph structure. Across all join sizes, local join completeness is zero. Structural and session completeness is 0.75 because the 16-input case exceeds the eight-record cap.

Figure 2 gives the resumption result. At 128 intervening records, session-wide recall is zero with contamination one; every task-aware policy retains recall one and contamination zero. The lifecycle control independently verifies that switching away from a hot 128-token wrong-task working set demotes all 128 tokens before promoting 64 active-task tokens. This is state-transition accounting, not a measured GPU transfer.

The scope–detail frontier in Figure 3 makes the two-axis decomposition operational. At selected detail, structural scope averages 0.875 recall using 168 charged tokens, versus 0.725 recall and 192 tokens for session scope. At native-K/V depth, structural scope reaches 0.641 recall at 192 tokens versus 0.241 for session scope. Local scope uses fewer tokens when the candidate partition is small, but its aggregate recall is 0.547 because joins require predecessor evidence. These estimates validate accounting and policy behavior; they are not measurements of model-native K/V byte transfer.

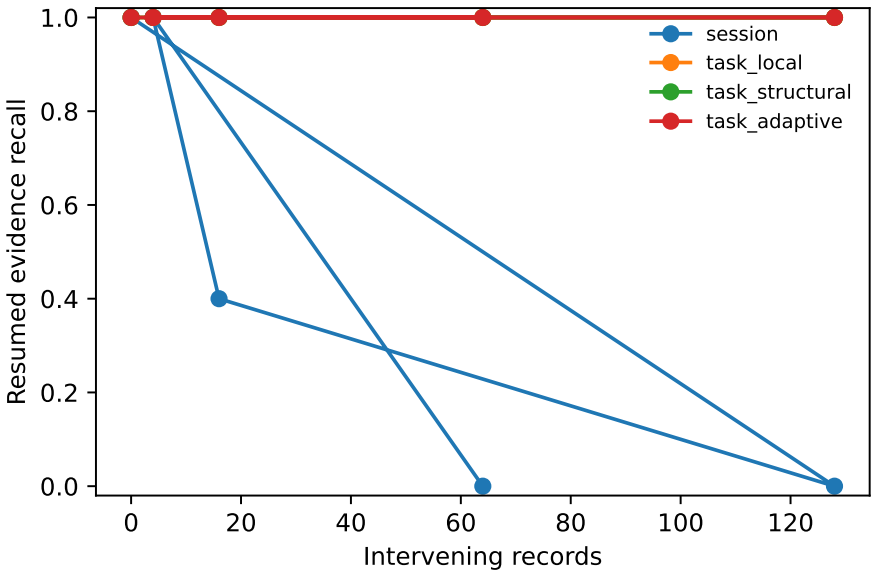

Figure 2: Resuming the oldest task after increasing intervening history. Task-local, structural, and adaptive scope preserve the required record; session-wide ranking is displaced by recent wrong-task evidence.

Table 2: Eight-input join, mean over five seeds. J is complete recovery of every predecessor input.

Policy	Recall	Precision	J
PRA_SESSION	1.000	1.000	1.00
PRA_TASK_LOCAL	0.125	0.167	0.00
PRA_TASK_STRUCT	1.000	1.000	1.00
PRA_TASK_ADAPTIVE	1.000	1.000	1.00

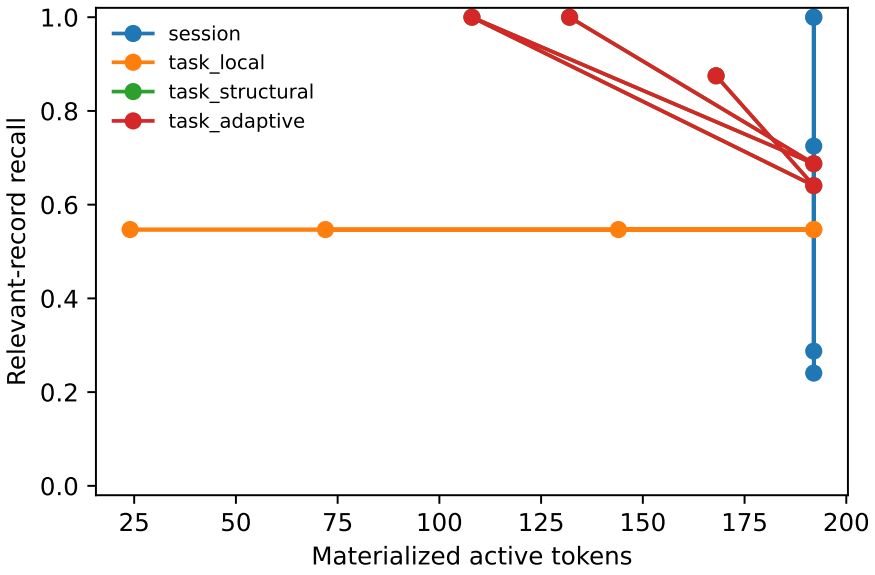

Figure 3: Scope breadth and materialization depth under a 192-token allowance. Structural scope spends the same or fewer charged tokens than session scope while recovering more relevant records at deeper views. Hard local scope is cheap but loses join inputs.

23 Production-PRA Confirmation

Protocol. The second experiment replaces the frozen lexical/recency control with the inherited Paper-7 production path. For each policy, task admission first produces record IDs; the native hybrid router then indexes the exact backing records, encodes the query, ranks original chunks, and requests selected native K/V under the same top-$K = 8$ setting. The model and router are frozen. We use Qwen/Qwen3-0.6B at revision `c1899de289a04d12100db370d81485cdf75e47ca`, deterministic decoding, and CUDA. The 15 cases cross low, medium, and high task confusability with independent, linear, join, resumption, and conflicting-state workflows. Their typed traces contain file reads, database results, terminal output, tool responses, and task state. The graph and active task are oracle harness inputs; task IDs are absent from the query.

We distinguish four events: required evidence was selected, its exact typed record was materialized, it was available to the model, and the answer consumed it correctly. The three unqualified PRA conditions use direct native memory. The `VISIBLE` diagnostic follows the same selection and exact backing path but serializes selected detail into the prompt. This is not a second router; it localizes failures after materialization. `FULL_SESSION`, `FULL_TASK_SCOPE`, and `COMPACT_ONLY` bound model behavior.

Table 3 confirms that the scope effect survives the stronger router. Session-wide routing has high record-level recall because it often selects one required chunk, yet only one third of cases have the complete evidence required by the answer. Structural closure admits dependencies, eliminates wrong-task records, and reaches 93.3% exact evidence availability with 60% fewer requested native tokens. Hard local scope is smaller but loses predecessor records, so its low contamination should not be mistaken for sufficient retrieval.

The evidence advantage grows from 40.0 percentage points at low confusability to 60.0 at medium and 80.0 at high (Figure 4). Visible-detail accuracy improves by 40, 60, and 60 points, respectively. With only five cases per level, the intervals are broad; the monotone evidence trend is a mechanism observation, not a precise scaling law.

The downstream result is positive but conditional. Full task scope exceeds the full session by 53.3 points, showing that unrelated interleaved context harms even when all evidence is present. Among routed visible-detail conditions, structural scope exceeds session scope by 53.4 points. Given available evidence, visible local and structural conditions answer correctly in 88.9% and 71.4% of cases. By contrast, all direct frozen native-memory conditions score zero, including structural cases in which evidence availability is 93.3%. The router found and materialized useful evidence; this model configuration did not consume it through direct memory injection. A learned integration gate or task-aware adaptation is therefore a separate requirement.

Physical capacity and graph geometry. The join sweep varies fan-in and encoded-ranking budgets independently over $\{2, 4, 8, 16, 32\}$. Fan-in 4 becomes retrieval-complete at $K = 8$, fan-in 8 at $K = 16$, and fan-in 16 at $K = 32$. At fan-in 32, even $K = 32$ recovers only 0.744 of predecessor records because active-task and chunk overhead also consume positions. Frozen-model join generation succeeds for fan-in 2 at $K \geq 4$ but fails for fan-in 4 and 8 even when retrieval is complete, again locating the failure in multi-record composition rather than scope alone.

Five independent DAG families—deep-narrow, shallow-wide, multi-fork, multi-join, and balanced—each run under five seeds. Session and structural scope have equal aggregate required-record recall (0.954), while structural scope removes all cross-task contamination. The matched result is useful: explicit structure changes the candidate set without manufacturing a retrieval advantage when the relevant graph geometry already fits the budget.

Accounting and replay. The production artifacts report serialized prompt tokens, encoded backing-chunk tokens, routing-index bytes, resident detail-cache bytes, indexing, query encoding, routing, and exact materialization time. These are measured runtime counters, although the Python orchestration is not an optimized serving benchmark. The real scope–detail frontier uses encoded chunk spans rather than the historical fixed depth costs. Persistent session replay reproduces task scope, selected record IDs, and typed provenance in every case while persisting no model cache. Parent edges participate in DAG cycle validation, including two-node and longer cycles.

Table 3: Production native routing, aggregated over 15 paired cases. R is required-record recall, C cross-task contamination, E exact evidence availability, and K_V requested native tokens. Only candidate scope changes.

Condition	R	C	E	K_V
PRA_SESSION	0.889	0.750	0.333	234.3
PRA_TASK_LOCAL	0.700	0.000	0.600	55.7
PRA_TASK_STRUCT	1.000	0.000	0.933	93.5

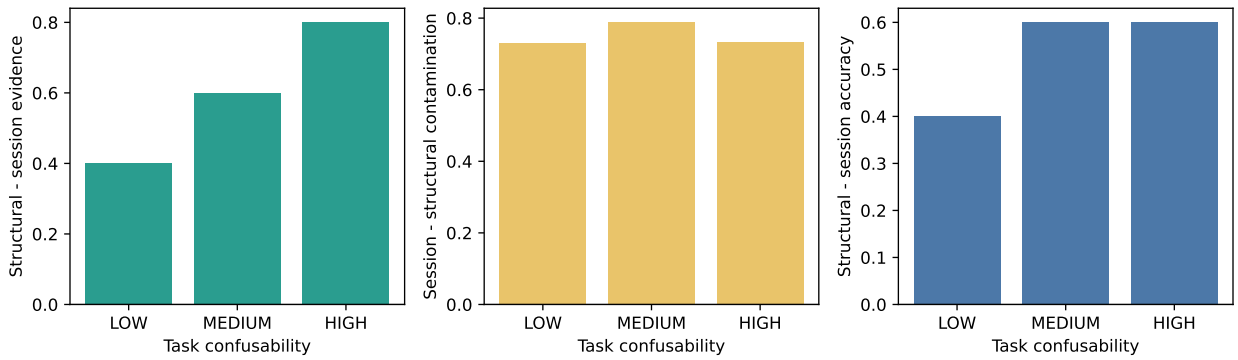

Figure 4: Paired structural-minus-session effects by semantic task confusability. Exact evidence and answer accuracy improve as task states become harder to distinguish lexically; session-wide contamination remains high. Each level contains five heterogeneous workflow cases, so intervals reflect case bootstrap uncertainty rather than a population-level benchmark estimate.

Table 4: Frozen-model answer accuracy and required-evidence use. Direct native-memory rows isolate consumption; visible rows expose the same selected exact records as serialized detail.

Condition	Accuracy	Evidence use
FULL_SESSION	0.200	0.433
FULL_TASK_SCOPE	0.733	0.800
COMPACT_ONLY	0.200	0.200
PRA_SESSION (native)	0.000	0.000
PRA_TASK_LOCAL (native)	0.000	0.000
PRA_TASK_STRUCT (native)	0.000	0.000
PRA_SESSION_VISIBLE	0.133	0.278
PRA_TASK_LOCAL_VISIBLE	0.533	0.633
PRA_TASK_STRUCT_VISIBLE	0.667	0.744

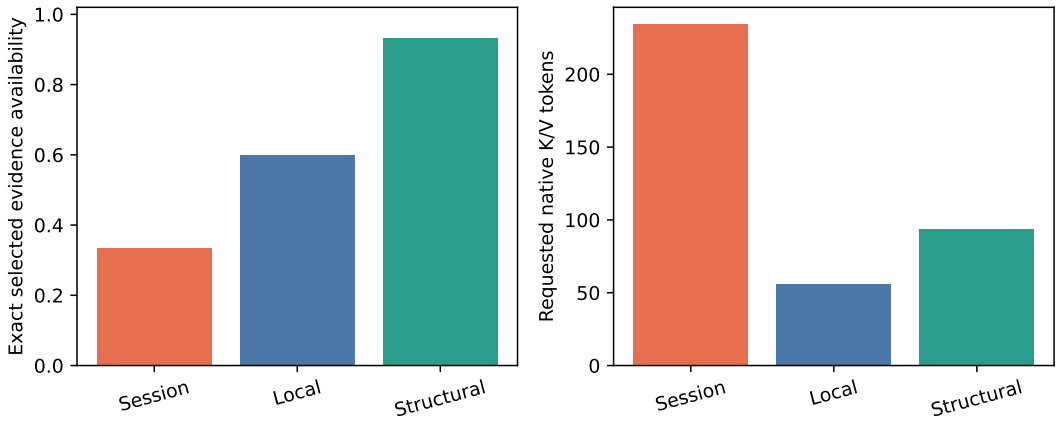

Figure 5: Production scope comparison using measured encoded chunks and index accounting. Structural scope improves exact evidence availability while requesting fewer native tokens than session-wide routing.

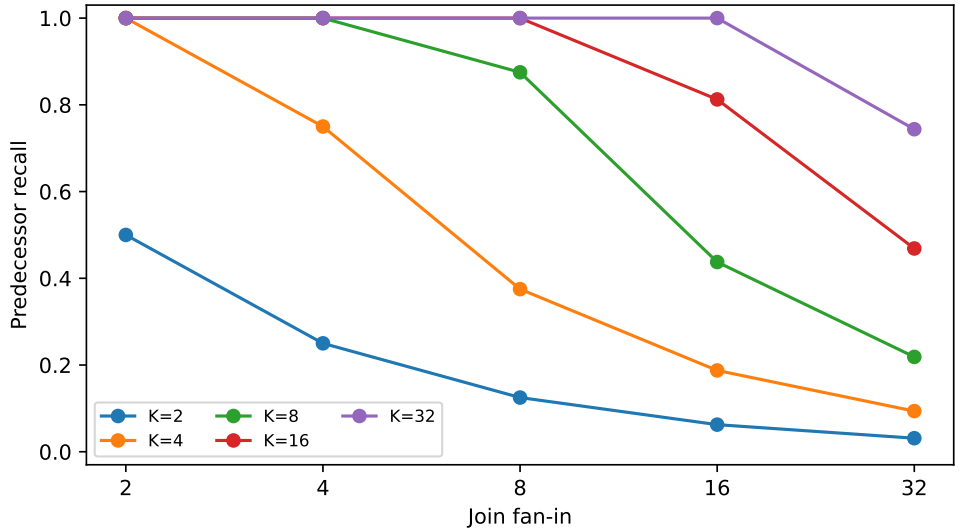

Figure 6: Predecessor recovery by join fan-in and encoded-chunk budget. Task structure removes unrelated competition but cannot override the physical selected-chunk budget.

24 Interpretation and Limits

The experiments establish that explicit task structure changes how a fixed PRA budget is allocated, that structural closure repairs the dependency loss caused by hard local filtering, and that the benefit survives a production native-Q/K router. They also establish downstream answer benefit when selected exact detail is consumable. The lexical/recency ranker is only the historical control.

Three limits matter. First, the production cohort is 15 controlled cases from one generator and one 0.6B model; five seeds are distributed across five workflow types rather than replicating every type. Second, direct native-memory consumption is negative, so the positive generation claim is about scoped selection plus exact selected-detail re-entry, not successful frozen memory injection. Third, structural scope cannot fit joins whose required chunks exceed the budget. Task scope removes irrelevant competition; it does not abolish capacity or composition limits.

Task identity and relations remain oracle. Metadata corruption, adaptive widening, model-produced preflight plans, online task tools, and hybrid acquisition are intentionally deferred until after this oracle production gate. JSON and Markdown parser fixtures validate syntax and graph invariants, but no model-generated plan is reported as evidence. Latencies include real encoding and materialization work on CUDA, yet exhaustive Python orchestration and a single workstation preclude serving claims.

25 Hypotheses and Status

H1. Interleaving degrades session-wide PRA faster than task-aware PRA. *Supported in the controlled scaling and production cohorts.*

H2. Explicit task scope reduces cross-task contamination without materially reducing required-evidence recall. *Supported for structural scope; hard local scope loses dependencies.*

H3. Gains increase with semantic similarity and hot-state contamination among tasks. *Evidence availability follows this trend; larger cohorts are required.*

H4. Structural closure outperforms hard task-local scope on dependency and join workflows while remaining cheaper than session-global retrieval. *Supported for routing; generation remains composition-limited at larger joins.*

H5. Explicit dependency relations outperform rediscovery of known dependencies.

H6. Task completion is an effective context-demotion boundary when compact result views retain full backing addressability.

H7. Task-conditioned working sets remain bounded/sublinear as total session history grows. *Supported by bounded cases, not established as an asymptotic law.*

H8. Task-aware gains increase with workflow width, depth, joins, resumption distance, and interleaving complexity.

H9. Preflight decomposition can recover much of the oracle task-aware gain for models that do not reliably manage tasks online.

H10. JSON and Markdown decomposition have model-dependent reliability; downstream utility matters more than exact graph match.

H11. Hybrid preflight-plus-online mutation is useful only if online changes add genuine execution-discovered structure rather than noise.

26 Experimental Discipline

The causal order is mandatory:

1. Given oracle task identity/relations, does task-aware scope improve PRA? *Passed for retrieval and selected-visible generation.*
2. How does the gain scale with workflow/interleaving complexity? *Bounded confusability, DAG, resumption, and join sweeps completed.*
3. How robust is it to imperfect/stale task metadata?
4. Can preflight decomposition recover useful structure?
5. Can online model-managed tasks match or improve preflight structure?
6. Does the hybrid justify its complexity?

Do not confound task extraction/planning competence with the core PRA mechanism.

27 Falsification

Weaken the thesis if session-wide PRA matches task-aware PRA at matched budget under strong interleaving; structural closure loses essential evidence or widens to near-global scope; scope/materialization bookkeeping costs approach the savings; completed-task demotion harms downstream execution; task-aware gains vanish on DAG/join workflows; metadata errors make the system too brittle; preflight decomposition costs more than it saves; generated task graphs do not improve downstream context selection; or gains occur only on synthetic task labels.

28 Implementation Status and Next Gates

Phase	Status	Artifact or remaining gate
A	Complete	Paper-7 records, views, backing, and cache lifecycle reused.
B	Complete	Versioned task graph, replay, provenance, and structural closure.
C	Complete	Four matched-budget scope policies with a frozen ranker.
D	Complete	Cold/host-encoded/device-hot states and measured Paper-7 counters separated.
E	Complete	Confusability, joins, independent DAGs, replay, and real frontier artifacts.
F	Complete	Oracle-graph frozen-model evaluation and evidence-consumption decomposition.
G	Deferred	Online task tools remain behind the oracle stop gate.
H	Deferred	Hybrid acquisition follows separate preflight and online evaluation.

29 Roadmap Boundary

Paper 8 remains single-session and does not introduce subagent context trees, cross-session memory, or a new general planner. Paper 9 extends the context topology to separate-session subagents. Paper 10 studies cross-session memory.

30 Target Claim

The paper does not claim invention of task lists, workflow DAGs, task planning, or decomposition prompting. Its target claim is:

Explicit typed task state and workflow relations can serve as a first-class context-scope signal, allowing production PRA to allocate fewer active retrieval tokens to more complete task evidence in an interleaved physical session. When selected exact detail is consumable, this improves downstream answers while keeping authoritative execution state in the harness.